identity provider, and customers logged in using short-lived OAuth links tied to specific projects. This narrow scope kept security simple: each request either came from an employee on a trusted device or from a preapproved customer identity already linked to a project record.

Inside that boundary, the platform used a ReBAC engine—first introduced in Section 6.1—to manage document sharing. Relationships such as owner, editor, and viewer were stored as graph data and evaluated consistently by the collaboration service. Those rules weren't the problem; they defined how people worked together on documents and were among the system's best features.

What wasn't built for growth was the authorization system behind the platform itself: the logic that determined who could sign in, which projects a customer could access, and what administrative actions an employee could perform. All of that code assumed a single organization—ACME—and was embedded in the application.

The main way to enforce these boundaries was through the project ID. When a new customer engagement started, ACME created a project record in its internal CRM; that record's auto-incrementing ID served as the link connecting everything else. Each document, folder, and comment stored the same `project_id`, and external customer accounts were provisioned with that ID as well. OAuth tokens for customer sessions also carried it. In practice, `project_id` acted like a substitute for tenancy, even though it was simply a database field shared across the system.

Authorization checks at the platform level depended solely on that value:

```
if request.user.company == "ACME" or request.user.project_id == proj
ect.id:
    allow_access(project)
```

That approach worked as long as there was only one security domain. Every identity was in ACME's directory, every project belonged to ACME, and each customer login was ultimately linked to an ACME account. However, it couldn't scale beyond that boundary. Once ACME considered turning this